Practical No. 4

Mapped CO: CO2 Units III & IV Bloom Level: L6 – Create

1.0) AIM

Develop and deploy an ERC-20 token using the Solidity programming language, perform token transfer operations, and inspect the contract's ABI, bytecode, and gas cost.

2.0) SCOPE

To understand how a standard fungible token is created on Ethereum using the ERC-20 standard, how tokens are transferred between accounts, and how a deployed contract is represented by its ABI and bytecode, along with the gas required for its operations.

3.0) FACILITIES / APPARATUS

- **Computer System:** Any desktop or laptop computer with an internet connection and a web browser.
- **Software:** Remix IDE (https://remix.ethereum.org). Optional for the extension: MetaMask + Ganache (from Practical 0), or MetaMask on the Sepolia testnet.

4.0) THEORY

What is a Token?

A token is a digital asset created and managed by a smart contract on a blockchain. Unlike ETH (the built-in currency of Ethereum), tokens are created by user-written contracts. Tokens can represent many things — a currency, reward points, a share, or a voting right.

Fungible vs Non-Fungible:

A fungible token is one where every unit is identical and interchangeable — like rupees, where one 10 note is worth exactly the same as any other. ERC-20 tokens are fungible. (Non-fungible tokens, or NFTs, are unique and are a different standard.)

What is the ERC-20 Standard?

ERC-20 (Ethereum Request for Comments 20) is a standard set of rules that a token contract must follow so that wallets, exchanges, and applications can all work with it in the same way. The standard defines common functions and information, including:

- name, symbol, and totalSupply — basic information about the token.
- balanceOf(account) — how many tokens an account holds.
- transfer(to, amount) — send tokens from your account to another.
- approve / transferFrom — allow another account to spend tokens on your behalf.

What is OpenZeppelin?

OpenZeppelin provides free, secure, community-audited implementations of standards like ERC-20. Instead of writing all the token logic ourselves (which is easy to get wrong), we import OpenZeppelin's tested ERC-20 contract and build on top of it. This makes our contract short, safe, and standard-compliant.

ABI, Bytecode, and Gas:

- Bytecode — when a Solidity contract is compiled, it becomes low-level bytecode. This is what is actually stored and executed on the blockchain by the Ethereum Virtual Machine (EVM).
- ABI (Application Binary Interface) — a JSON description of the contract's functions and events. Applications and wallets use the ABI to know how to call the contract.
- Gas — every operation on Ethereum costs "gas". Deploying a contract and writing data (like a transfer) cost more gas than simply reading data, because they change the blockchain's state.

5.0) PROGRAM (Solidity — MyToken.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

// Import OpenZeppelin's ready-made, audited ERC-20 contract
import "https://github.com/OpenZeppelin/openzeppelin-
contracts/blob/v5.0.0/contracts/token/ERC20/ERC20.sol";

contract MyToken is ERC20
{
    // Give the token a name ("MyCollegeToken") and a symbol ("MCT")
    constructor(uint256 initialSupply) ERC20("MyCollegeToken", "MCT") {
        // Create the initial supply and give it to the deployer
        _mint(msg.sender, initialSupply * (10 ** decimals()));
    }
}
```

Note: *ERC-20 tokens use 18 decimal places (like ETH). The (10 ** decimals()) part converts a simple supply number, e.g. 1000, into the correct on-chain amount.*

6.0) PROCEDURE / STEPS TO RUN

1. **Open Remix:** Go to https://remix.ethereum.org.
2. **Create the file:** In the 'contracts' folder, create MyToken.sol and paste the program above. (Remix automatically downloads the OpenZeppelin import.)
3. **Compile:** On the 'Solidity Compiler' tab, choose compiler version 0.8.20 or later and click 'Compile MyToken.sol'.
4. **Deploy:** On the 'Deploy & Run Transactions' tab, set ENVIRONMENT to 'Remix VM (Cancun)'. Next to Deploy, enter an initial supply (e.g., 1000) and click 'Deploy'.
5. **Check your balance:** Under 'Deployed Contracts', copy your account address (from the ACCOUNT box at the top) and paste it into balanceOf, then click it. You should hold the full supply.
6. **Transfer tokens:** Select a second account from the ACCOUNT dropdown and copy its address. Switch back to your first account, and in transfer enter that address and an amount (e.g., 100 followed by 18 zeros), then click 'transfer'.
7. **Verify balances:** Call balanceOf for both accounts to confirm the sender's balance decreased and the recipient's increased.
8. **Inspect ABI & bytecode:** Go back to the 'Solidity Compiler' tab and click 'Compilation Details' (or the ABI and Bytecode copy icons) to view the contract's ABI (JSON) and bytecode.
9. **Observe gas:** In the Remix console (bottom), expand the deploy and transfer transactions and note the 'transaction cost / gas' value for each. Compare them.

7.0) EXPECTED OUTPUT

After deployment, the MyToken contract appears under 'Deployed Contracts' with ERC-20 functions (balanceOf, transfer, totalSupply, etc.). balanceOf initially shows the full supply for the deployer; after a transfer, the balances change accordingly.

```
totalSupply()            -> 100000000000000000000   (1000 tokens, 18 decimals)
balanceOf(deployer)      -> 100000000000000000000

transfer(account2, 100 * 10^18)  -> transaction successful

balanceOf(deployer)      -> 90000000000000000000    (900 tokens)
balanceOf(account2)      -> 10000000000000000000    (100 tokens)
```

Observation: *Deploying the contract costs the most gas (several hundred thousand), a transfer costs far less (roughly 40,000–65,000 gas), and balanceOf is free because it only reads data. The ABI is a JSON list of the token's functions; the bytecode is the compiled code stored on-chain.*

Optional Extension — Deploy to a Real Network and See the Token in MetaMask

The steps above run on Remix's test blockchain. To deploy through MetaMask so the token appears in a real wallet, use either Ganache (local) or the Sepolia public testnet:

1. Ganache (local): Start Ganache and connect MetaMask to the Ganache Local network (Practical 0). Sepolia (public): switch MetaMask to Sepolia and get test ETH from a faucet such as https://www.alchemy.com/faucets/ethereum-sepolia.
2. In Remix, set ENVIRONMENT to 'Injected Provider – MetaMask' and approve the connection.
3. Enter the initial supply and click 'Deploy'; confirm the transaction (and gas fee) in the MetaMask popup.
4. Copy the deployed contract address from Remix. In MetaMask, go to Tokens → 'Import tokens', paste the contract address, and the token symbol (MCT) fills automatically. Import it.
5. Your MCT token balance now appears in MetaMask. On Sepolia, you can also view the contract and transactions on https://sepolia.etherscan.io by searching the contract address.

Tip: Sepolia faucets are often slow or rate-limited. For a reliable classroom experience, the Ganache option is recommended; use Sepolia only if faucet test ETH is available.

8.0) CONCLUSION

In this way, we successfully developed and deployed an ERC-20 fungible token using Solidity and OpenZeppelin, performed a token transfer between accounts, and inspected the contract's ABI, bytecode, and gas costs — demonstrating how standard tokens are created and how deployed smart contracts are represented and executed on Ethereum.